

EXECPLAN — Technical Report

Version: MVP 1.0 · **Date:** June 2026 · **Status:** All four build phases complete and verified

1. Executive summary

EXECPLAN is an operational plan activation and live execution tracking system. It is deliberately not a general project-management tool: the entire product is organized around one cycle — create plan → activate → notify people → execute tasks → live dashboard. A plan manager defines a reusable plan (teams, task templates, shift roster); activating it freezes a snapshot, notifies everyone on duty in the current shift, generates their tasks, and opens a live dashboard that the whole organization watches. Non-responders are escalated through repeated call attempts until their pre-resolved substitutes are pulled in automatically.

The MVP comprises three deliverables sharing one backend: a Django REST API, an Arabic-first Next.js web application for managers and administrators, and an Expo (React Native) mobile application for field roles. The complete cycle has been verified end-to-end against the running system: activation resolves participants from the live roster, a member acknowledges and completes tasks from the mobile flow, the dashboard reflects it within one 10-second polling interval, and five unanswered call attempts trigger automatic substitute induction.

2. Architecture overview

The system is a classic three-tier architecture with one deliberate twist: all domain logic lives in a service layer inside the backend, so the API, the CLI, and any future scheduler trigger identical behavior.

The backend is Django 5 with Django REST Framework over PostgreSQL, exposing a versioned API under `/api/v1/`. Authentication is JWT (access plus refresh tokens) via `django-rest-framework-simplejwt`. The web client is Next.js 14 (App Router) with Tailwind CSS, reaching the backend through a same-origin rewrite (`/backend/*` → Django), which eliminates CORS configuration entirely. The mobile client is Expo with TypeScript, talking to the backend directly over the LAN or internet with the same JWT flow and AsyncStorage persistence.

Real-time behavior is intentionally simple in the MVP: every client polls a single aggregated dashboard endpoint every 10 seconds. There are no WebSockets, no Django Channels, no Celery, and no Redis. This was a locked decision and it pays off operationally — the entire system runs with two processes (Django and the Next server) plus Postgres, which matters for the closed or modest infrastructure environments typical of municipal and emergency-services customers. The polling endpoint performs all aggregation server-side in one round trip, so each poll is one cheap request per connected screen.

The timezone is Asia/Kuwait throughout, applied at the Django settings level so shift calculations and timestamps are consistent across server, web, and mobile.

3. Domain model

The central design decision is the separation of **Plan** (a reusable template) from **PlanActivation** (an immutable runtime snapshot). A plan holds metadata, teams, task templates, and references to a roster. Activation copies everything that matters at that instant: the participants resolved from the shift roster, their team names as plain text snapshots, and execution tasks cloned from the templates with computed due times. Editing a plan afterwards can never disturb a running activation, because the activation no longer depends on the plan's mutable structure.

Fifteen models implement the MVP. User is a custom auth model carrying a role (system admin, plan manager, team leader, team member) plus phone and organizational placement. Organization and Department provide the hierarchy. Plan, Team, TeamMembership, and TaskTemplate form the template side. ShiftAssignment is the roster: a row of (user, team, shift, date, is_substitute_for), where a non-null `is_substitute_for` marks the row as someone's designated substitute for that shift. On the runtime side sit PlanActivation, ActivationParticipant, ExecutionTask, NotificationLog, CallAttempt, ResponseStatus, EscalationLog, and BroadcastMessage.

Three fixed shifts cover the day: 06:00–14:00, 14:00–22:00, and 22:00–06:00. A night shift in progress after midnight resolves to the previous calendar day's roster, since the 22:00–06:00 shift belongs to the day it started. A shift definition alone cannot answer "who is on duty now"; the ShiftAssignment table answers it, and activation resolves the answer once and freezes it. If an activation crosses a shift boundary, participants remain those frozen at activation time — automatic handover is documented as a future feature.

4. Activation and escalation flow

Activation is a single atomic transaction. It refuses to start if the plan already has an active run or if no one is on duty for the current shift. Otherwise it creates the activation record, resolves on-duty participants per team from today's roster, records each participant's

substitute (resolved from the `is_substitute_for` chain and frozen on the participant row), generates one `ExecutionTask` per participant per task template of his team with a due time derived from the template's duration, writes one in-app `NotificationLog` per participant, and logs call attempt #1 per participant.

A "response" has exactly one definition: the participant tapped the Acknowledge button («أنا جاهز») in the app, which creates a `ResponseStatus` row and flips his status to `RESPONDED`. Nothing else counts.

Escalation is manually triggered in the MVP — a dashboard button and an equivalent management command (`python manage.py run_escalation`) run the same service function. Each cycle adds one `CallAttempt` for every participant still pending. When a participant reaches five attempts without responding, the cycle writes an `EscalationLog`, marks him `ESCALATED`, and inducts his frozen substitute as a new participant — with his own copy of the team's tasks, his own notification, and his own call attempt #1. The threshold lives in settings (`ESCALATION_MAX_ATTEMPTS`), and the trigger is designed to move behind a scheduler later without touching the logic, since the API action and the CLI command are thin wrappers over one function.

The dashboard endpoint returns everything a monitoring screen needs in one JSON document: required, responded, non-responded, escalated, and substitutes-called counts; response and task-completion rates; per-team progress sorted best-first (which directly yields the "best team" and "delayed team" badges); overdue tasks; and a merged, chronologically sorted event feed of the latest fifty events synthesized from the notification, call, response, escalation, task-completion, and broadcast tables — no separate event table is needed.

5. API surface and security model

All endpoints sit under `/api/v1/`. Authentication endpoints issue and refresh JWTs. Standard CRUD covers organizations, departments, users, plans, teams, team memberships, shift assignments, and task templates. The activation lifecycle is expressed as actions: activate on a plan; then acknowledge, run-escalation, dashboard, broadcast, close, my-tasks, my-notifications, set-substitute, and raise-issue on an activation. Execution tasks support list and PATCH only — marking done, adding a note, and (for leaders and managers) reassigning to another participant.

Authorization is enforced twice: DRF permission classes gate each endpoint by role, and every list queryset is filtered by the requester's role, so a team member can never even enumerate tasks or activations that are not his. Leaders see their teams; managers and admins see everything. Reassignment carries object-level checks — a leader can move tasks only between participants of teams he leads (verified to return 403 across team

boundaries), and a serializer-level validation rejects moving a task to a participant of a different activation. Activation rights are restricted to a plan's creator, its explicitly authorized activators, and admins. Three findings from testing were fixed during development: trailing-slash handling to match the specified URL style, queryset deduplication where role-scoped joins produced duplicate rows, and the read-only user listing granted to plan managers so the wizard can populate team membership choices while writes remain admin-only.

6. Notification abstraction

The MVP sends nothing externally by design — notifications and calls are database placeholders. What matters architecturally is the seam: a `NotificationProvider` abstract interface with exactly two methods, `send_notification` and `place_call`, currently implemented by a `DatabasePlaceholderProvider` that writes `NotificationLog` and `CallAttempt` rows. All callers obtain the provider through a single factory function. Integrating Twilio voice, Infobip SMS, or the WhatsApp Business API later means writing one new class and changing one line; no activation, escalation, or broadcast code is touched.

7. Frontend engineering

Both clients share a design language built for non-technical operators: an operations-room navy for chrome, amber reserved for consequential actions (plan launch), green for acknowledgment and completion, red for escalation and overdue states, and one oversized action per screen. The typeface is IBM Plex Sans Arabic, bundled locally through npm so the system renders correctly on networks with no access to font CDNs.

The web app implements login, admin tables for users and departments, the manager's plan list, a strictly four-step creation wizard (plan details, teams and members, tasks, shifts and review) with per-step Arabic validation, a plan page dominated by the launch button behind a plain-language confirmation, and the live dashboard. When an activation closes, the dashboard route renders a final summary of counts only and stops polling. The mobile app routes by role after login: members get the acknowledge button, their task checklist with notes, and a broadcast inbox; leaders get team progress, task reassignment, substitute management, and an issue-reporting box that surfaces in the manager's event log; managers get a mini live dashboard with escalation, broadcast, and close actions above their plans.

Internationalization is scaffolded, not just promised: both apps carry complete Arabic and English locales in single translation files, with a working language toggle that flips text direction on the web and switches instantly on mobile. Adding a third language is one additional object with the same keys.

8. Verification

Every phase closed against its acceptance criteria on the running system rather than by inspection. Phase 1 was exercised over HTTP for all four roles: activation snapshotted the correct on-duty participants, member-only task visibility held, acknowledgment flowed, four escalation cycles drove non-responders to the five-attempt threshold and inducted the rostered substitute, and the dashboard JSON returned correct aggregates throughout. Phase 2 was verified by a clean production build and a live proxy test covering login, the wizard's data dependencies, activation, and the dashboard payload and page. Phase 3, lacking an emulator in the build environment, was verified by a clean strict type-check plus a replay of every mobile screen's exact API sequence — including the acceptance criterion that a member's acknowledgment and task completion appear in the manager's dashboard payload within one polling interval. Phase 4 verified the broadcast-to-inbox chain and the close-to-summary transition. Known residual risk: physical-device layout (spacing, RTL edge cases) has not been exercised and will need a polish pass on first Expo Go runs.

9. Deployment topology and operations

Local development needs only docker-compose for Postgres plus `runserver` and `expo start` ; a zero-dependency SQLite mode exists for quick evaluation. Production targets a single internet-connected VPS or cloud instance: gunicorn serving Django behind nginx, the Next.js production server (or static export) on the same host, Postgres locally or managed, and the Expo app distributed through EAS builds or Expo Go during piloting, pointed at the server via a public URL. Escalation can be driven by cron invoking the management command until a scheduler is introduced. Secrets and database credentials are environment-driven with a committed `.env.example` .

10. Limitations and roadmap

The MVP's consciously deferred items form the technical roadmap: real notification channels behind the existing provider interface; timed automatic escalation via a scheduler or Celery beat; shift handover when activations cross shift boundaries; surveys, lessons learned, and the full FinalReport; checklist granularity below the task level; permission granularity beyond roles; and a move from polling to push (WebSockets or mobile push notifications) when scale demands it. None of these requires re-architecture — each lands on a seam the MVP deliberately left open.